Usage of 2D Line Plot

Usage in Research and Projects

Line Plot হল এমন একটি visualization টুল যা data-র ধারাবাহিক পরিবর্তন দেখাতে এবং তার ট্রেন্ড বিশ্লেষণ করতে সাহায্য করে। বাস্তব গবেষণা ও প্রজেক্টে এর প্রয়োগ অনেক বিস্তৃত এবং গুরুত্বপূর্ণ। নিচে আমরা বিভিন্ন ক্ষেত্র অনুযায়ী এর ব্যবহার বিস্তারিতভাবে আলোচনা করব।

1. Scientific Research

গবেষণামূলক কাজে line plot ব্যবহৃত হয় সময়ের সঙ্গে সঙ্গে কোনো পরিমিতির পরিবর্তন বুঝতে। উদাহরণস্বরূপ:

- Temperature change over time in a chemical reaction.
- Growth rate of bacteria in a lab experiment.
- Performance or accuracy improvement across experimental trials.

এমন ক্ষেত্রে line plot দ্রুত বুঝতে সাহায্য করে কোন শর্তে উন্নতি হচ্ছে এবং কোথায় স্থিতিশীলতা এসেছে।

2. Business and Financial Analysis

ব্যবসা বা অর্থনীতির ক্ষেত্রে line plot ব্যবহৃত হয় বাজারের পরিবর্তন বা বিক্রির প্রবণতা বোঝার জন্য। যেমন:

- Monthly or yearly **sales growth** visualization.
- Stock price movement analysis over a certain period.
- Profit or revenue comparison among products or regions.

এই visualization decision-making এবং forecasting-এ বড় ভূমিকা রাখে।

3. Machine Learning and Data Science

Machine Learning model training-এর সময় line plot অনেক গুরুত্বপূর্ণ।

- Training vs validation accuracy বা loss ট্র্যাক করতে line plot ব্যবহার করা হয়।
- Model performance improvement visualization সহজ হয়।
- Hyperparameter tuning ফলাফল তুলনা করতেও এটি কার্যকর।

4. Engineering and Sensor Data Monitoring

Engineering projects-এ sensor data বা performance metrics visualization করতে line plot প্রায়ই ব্যবহৃত হয়। যেমন:

- Speed vs time, pressure vs temperature, বা current vs voltage relationship।
- Continuous monitoring of system performance in control systems।

5. Academic and Educational Uses

শিক্ষাক্ষেত্রে data analysis শেখানোর সময় line plot প্রথমে শেখানো হয় কারণ এটি সহজ, visual এবং direct। শিক্ষার্থীরা এটি ব্যবহার করে data trends বুঝতে পারে।

Conclusion

Line plot একটি basic কিন্তু অত্যন্ত powerful visualization method যা বিভিন্ন ক্ষেত্রে data trends এবং পরিবর্তনকে সহজে প্রকাশ করে। গবেষণা, ব্যবসা, প্রযুক্তি, বা শিক্ষাক্ষেত্র — সবখানেই এটি গুরুত্বপূর্ণ ভূমিকা পালন করে।

2D Line Plot Basics

Introduction

এই lesson-এ আমরা 2D Line Plot এর মৌলিক কাঠামো এবং বৈশিষ্ট্য শিখব। Line plot data visualization-এর সবচেয়ে সাধারণ ফর্মগুলোর একটি, যা একটি ধারাবাহিক পরিবর্তনকে প্রদর্শন করে। এখানে আমরা color, style, এবং multiple line plotting সম্পর্কেও জানব।

Step-by-Step Explanation

Step 1: Import Required Library

প্রথমে Matplotlib লাইব্রেরির pyplot module import করতে হবে।

```
import matplotlib.pyplot as plt
```

Step 2: Prepare Data

আমরা দুটি dataset ব্যবহার করব যাতে multiple lines একসাথে plot করা যায়।

```
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]
y2 = [1, 3, 5, 7, 9]
```

Step 3: Plot Lines

`plt.plot()` function ব্যবহার করে একাধিক লাইন plot করা যায়। এখানে আমরা color, linestyle, এবং marker ব্যবহার করে প্রতিটি লাইনকে আলাদা করব।

```
plt.plot(x, y1, color='blue', linestyle='-', marker='o', label='Line 1')
plt.plot(x, y2, color='red', linestyle='--', marker='s', label='Line 2')
```

Step 4: Add Title, Labels, and Legend

Plot বুঝতে সহজ করার জন্য title, axis labels, এবং legend যোগ করা জরুরি।

```
plt.title('2D Line Plot Basics')
plt.xlabel('X Axis - Values')
plt.ylabel('Y Axis - Measurements')
plt.legend()
```

Step 5: Add Grid and Show Plot

Grid plot কে আরও পরিষ্কার করে এবং points align করতে সাহায্য করে।

```
plt.grid(True)
plt.show()
```

Full Example Code

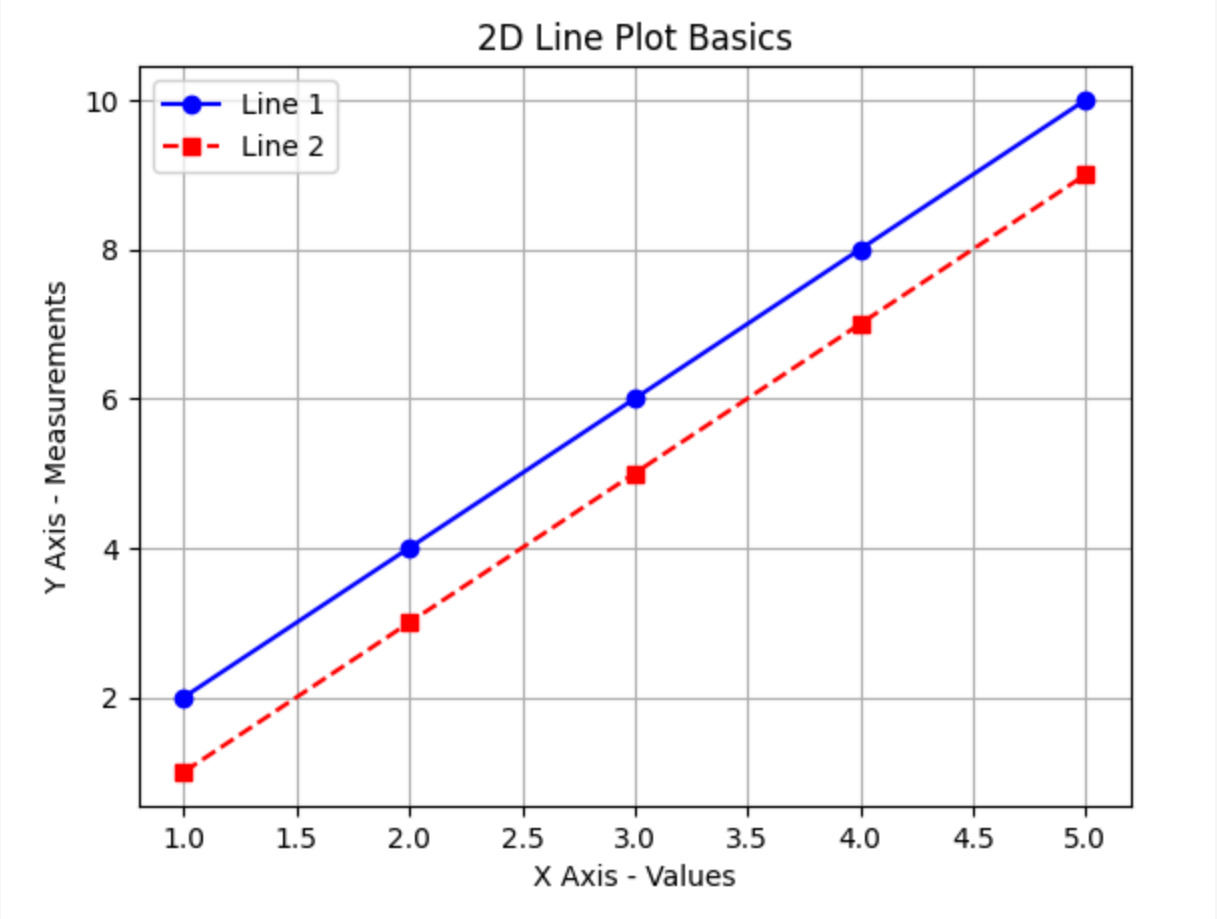
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]
y2 = [1, 3, 5, 7, 9]

plt.plot(x, y1, color='blue', linestyle='-', marker='o', label='Line 1')
plt.plot(x, y2, color='red', linestyle='--', marker='s', label='Line 2')

plt.title('2D Line Plot Basics')
plt.xlabel('X Axis - Values')
plt.ylabel('Y Axis - Measurements')
plt.legend()
plt.grid(True)
plt.show()
```

Output



Output Explanation

এই plot-এ আপনি দুটি লাইন দেখতে পাবেন: প্রথমটি নীল রঙে এবং দ্বিতীয়টি লাল dashed স্টাইলে। marker দিয়ে প্রতিটি data point স্পষ্টভাবে দেখা যায়। legend plot-এর প্রতিটি লাইন কোন dataset নির্দেশ করছে তা বুঝতে সাহায্য করে। grid যোগ করায় graph আরও পরিষ্কার এবং professional দেখায়।

2D Line Plot from Pandas DataFrame

Introduction

এই lesson-এ দেখানো হয়েছে কীভাবে **Pandas DataFrame** থেকে সরাসরি **2D Line Plot** তৈরি করা যায়। বাস্তব জীবনের dataset সাধারণত CSV, Excel, বা database থেকে আসে, এবং Pandas ব্যবহার করে সেগুলো সহজভাবে load, clean এবং visualize করা সম্ভব।

Line plot এমন একটি visualization যা সময়ের সঙ্গে data-র পরিবর্তন প্রদর্শন করে। যখন data column আকারে থাকে, তখন Pandas DataFrame plotting method ব্যবহারে কোড সংক্ষিপ্ত ও বোধগম্য হয়।

Step-by-Step Explanation

Step 1: Import Required Libraries

Data handling এর জন্য **Pandas** এবং visualization এর জন্য **Matplotlib** import করা হয়।

```
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Create or Load a DataFrame

Data manually তৈরি করা যেতে পারে অথবা CSV ফাইল থেকে read করা যেতে পারে।

```
# Example 1: Creating DataFrame manually
data = {
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
    'Sales_A': [220, 270, 300, 350, 400, 420],
    'Sales_B': [180, 230, 280, 310, 360, 390]
}
df = pd.DataFrame(data)

# Example 2: Reading from CSV file
# df = pd.read_csv('sales_data.csv')
```

এই DataFrame-এ প্রতিটি row একটি মাসকে এবং প্রতিটি column একটি product-এর sales value নির্দেশ করে।

Step 3: Plot Using DataFrame

DataFrame-এর built-in `plot()` method Matplotlib-এর উপর ভিত্তি করে কাজ করে।

```
df.plot(x='Month', y=['Sales_A', 'Sales_B'], kind='line', marker='o', linewidth=2)
```

`x` parameter X-axis এবং `y` parameter multiple columns নির্ধারণ করতে ব্যবহৃত হয়।

Step 4: Customize the Plot

Plot-কে আরও স্পষ্ট এবং উপস্থাপনযোগ্য করার জন্য customization প্রয়োগ করা হয়।

```
plt.title('Monthly Sales Comparison between Product A and B')
plt.xlabel('Month')


plt.ylabel('Sales (in Units)')
```

```
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(['Sales_A', 'Sales_B'], loc='upper left')
plt.tight_layout()
```

Step 5: Display the Plot

```
plt.show()
```

Advanced Example: Adding Styles and Colors

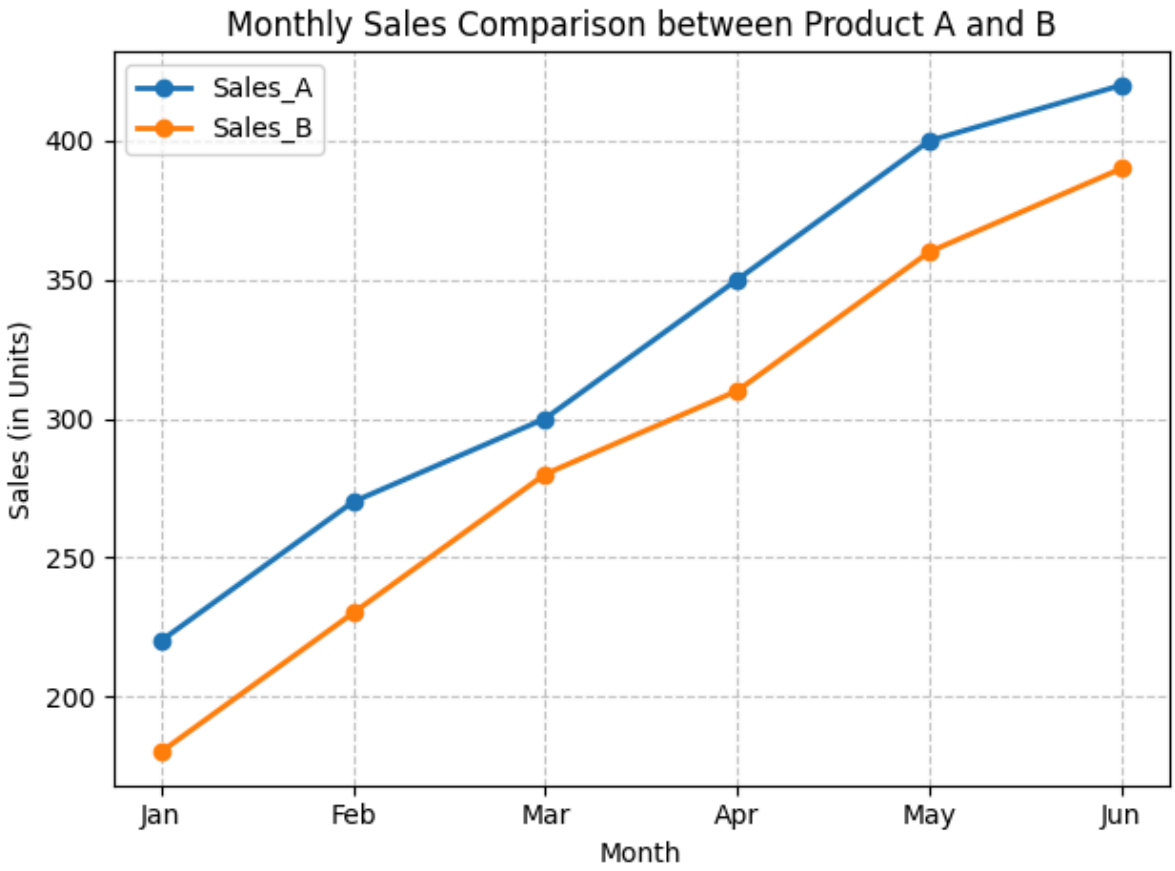
Plot style পরিবর্তন এবং visual enhancement-এর জন্য predefined styles ও formatting ব্যবহার করা হয়।

```
plt.style.use('seaborn-v0_8-darkgrid')

ax = df.plot(x='Month', y=['Sales_A', 'Sales_B'], marker='o', linewidth=3)
ax.set_title('Sales Performance - Styled Plot', fontsize=14, fontweight='bold')
ax.set_xlabel('Month', fontsize=12)
ax.set_ylabel('Sales (Units)', fontsize=12)
ax.legend(['Sales_A', 'Sales_B'], title='Product')
plt.show()
```

এই উদাহরণে **seaborn style**, **font customization**, এবং **legend title** ব্যবহারের মাধ্যমে corporate বা academic presentation উপযোগী chart প্রদর্শিত হয়।

Output



Output Explanation

Plot-এ দুটি লাইন দেখা যায় — **Sales_A** এবং **Sales_B**। প্রতিটি মাসে দুই product-এর বিক্রির পার্থক্য স্পষ্টভাবে বোঝা যায়। Grid ও marker যোগ করায় plotটি আরও visually পরিষ্কার হয় এবং মান নির্ধারণ সহজ হয়।

Practice Question

Task:

A dataset contains monthly temperature data for two cities — City A and City B.

Create a Pandas DataFrame using the following data:

```
1data = {
2    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
3    'City_A': [22, 24, 28, 33, 36, 38],
4    'City_B': [18, 20, 25, 30, 34, 35]

5}
```

Plot both temperature lines on the same graph using `df.plot()`.

Add title, labels, legend, and grid.

Experiment with color, line style, and markers.

Challenge:

Try to find in which month the temperature difference between the two cities is maximum using Pandas methods, and highlight that point in the plot.

Customizing Line Plot (Color, Linestyle, and Marker)

Introduction

এই lesson-এ আলোচনা করা হয়েছে কীভাবে **Matplotlib Line Plot** আরও কাস্টমাইজ করা যায়। Line plot শুধু data visualization নয়, বরং presentation-এর অংশও। তাই line-এর **color**, **linestyle**, এবং **marker** কাস্টমাইজ করলে plot আরও স্পষ্ট ও আকর্ষণীয় হয়।

Step-by-Step Explanation

Step 1: Import Required Library

```
1import matplotlib.pyplot as plt
```

Step 2: Prepare Data

```
1x = [1, 2, 3, 4, 5]
2y1 = [2, 4, 6, 8, 10]
3y2 = [1, 3, 5, 7, 9]
```

Step 3: Customize Line Color

Line color নির্ধারণের জন্য `color` parameter ব্যবহার করা হয়।

```
1plt.plot(x, y1, color='blue', label='Blue Line')
2plt.plot(x, y2, color='red', label='Red Line')
```

Step 4: Change Linestyle

Line style পরিবর্তনের জন্য `linestyle` parameter ব্যবহৃত হয়।

```
1plt.plot(x, y1, color='blue', linestyle='-', label='Solid Line')
```

```
2plt.plot(x, y2, color='red', linestyle='--', label='Dashed Line')
```

Common linestyles:

- `'-'` → solid line
- `'--'` → dashed line
- `'-.'` → dash-dot line
- `':'` → dotted line

Step 5: Add Markers

Markers data points নির্দেশ করতে ব্যবহৃত হয়।

```
1plt.plot(x, y1, color='blue', marker='o', label='Circle Marker')
2plt.plot(x, y2, color='red', marker='s', label='Square Marker')
```

Some common markers:

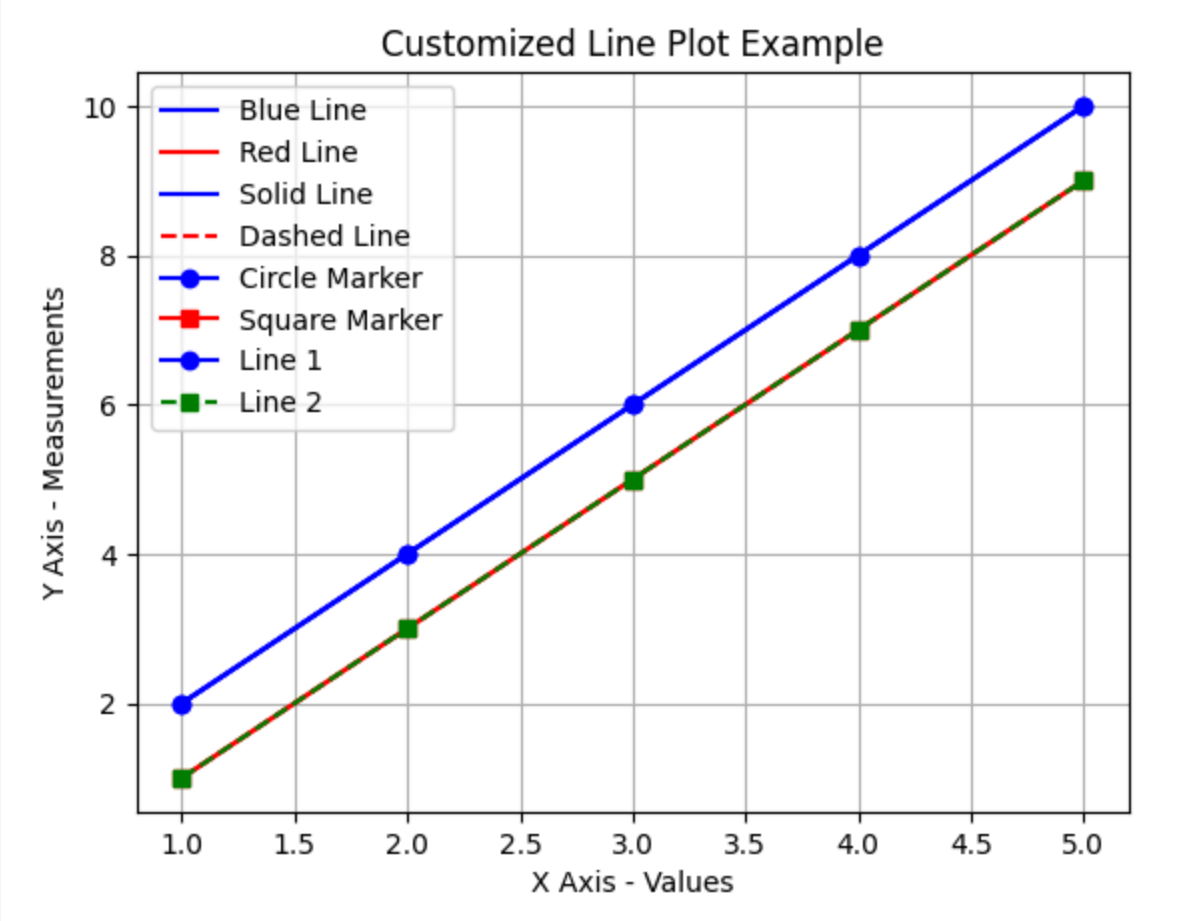
- `'o'` → circle
- `'s'` → square
- `'^'` → triangle_up
- `'D'` → diamond

Step 6: Combine All Customizations

সব কাস্টমাইজেশন একত্রে ব্যবহার করে একটি সম্পূর্ণ plot তৈরি করা যায়।

```
1plt.plot(x, y1, color='blue', linestyle='-', marker='o', label='Line 1')
2plt.plot(x, y2, color='green', linestyle='--', marker='s', label='Line
2')
3
4plt.title('Customized Line Plot Example')
5plt.xlabel('X Axis - Values')
6plt.ylabel('Y Axis - Measurements')
7plt.legend()
8plt.grid(True)
9plt.show()
```

Output



Output Explanation

এই plot-এ দুটি লাইন দেখা যাবে, প্রতিটিতে আলাদা color, linestyle এবং marker ব্যবহার করা হয়েছে। এর ফলে data point গুলো স্পষ্টভাবে চিহ্নিত করা যায় এবং দুটি dataset-এর পার্থক্য পরিষ্কারভাবে বোঝা যায়।

Practice Question

Task:

A dataset contains weekly rainfall (in mm) for two cities. Create two lists:

```
1week = [1, 2, 3, 4, 5, 6]
2city1 = [15, 22, 18, 30, 25, 28]
3city2 = [10, 14, 20, 24, 27, 29]
```

Plot both rainfall trends on the same graph using different colors, linestyles, and markers.

Challenge:

Add gridlines, labels, legend, and title. Highlight the highest rainfall week for both cities with a different marker or annotation.

Usage of Scatter Plots

Introduction

এই lesson-এ আলোচনা করা হয়েছে **Scatter Plot** নিয়ে — এটি data visualization-এর একটি গুরুত্বপূর্ণ পদ্ধতি যা দুটি ভেরিয়েবলের মধ্যে সম্পর্ক বোঝাতে ব্যবহৃত হয়। **Scatter Plot** ব্যবহার করে correlation, pattern, এবং outlier চিহ্নিত করা যায়।

Theory

Scatter plot হল এমন একটি visualization টুল যেখানে প্রতিটি point একটি observation বা data pair নির্দেশ করে। X-axis সাধারণত independent variable এবং Y-axis dependent variable বোঝায়। এটি continuous numerical data-র মধ্যে সম্পর্ক বা trend বোঝাতে অত্যন্ত কার্যকর।
Correlation এর ধরণ সাধারণত তিন প্রকার:

> **Positive Correlation:** X বাড়লে Y-ও বাড়ে।
> **Negative Correlation:** X বাড়লে Y কমে।
> **No Correlation:** X ও Y-এর মধ্যে কোনো সম্পর্ক থাকে না।

Scatter plot data distribution, clustering এবং outlier সনাক্ত করার জন্যও ব্যবহৃত হয়।

Example

```
import matplotlib.pyplot as plt

# Example data
temperature = [22, 24, 26, 28, 30, 32, 34, 36]
ice_cream_sales = [110, 130, 150, 170, 200, 230, 260, 290]

# Create Scatter Plot
plt.scatter(temperature, ice_cream_sales, color='orange', s=80, alpha=0.8)


plt.title('Temperature vs Ice Cream Sales')
plt.xlabel('Temperature (°C)')
```

```
plt.ylabel('Sales (Units)')
plt.grid(True)
plt.show()
```

এই plot থেকে দেখা যায় যে temperature বাড়ার সাথে সাথে ice cream sales ও বাড়ছে — এটি একটি **positive correlation**।

Fields of Use

Scatter plot বিভিন্ন ক্ষেত্রেই ব্যাপকভাবে ব্যবহৃত হয়:

- **Scientific Research:** দুটি ভেরিয়েবলের মধ্যে সম্পর্ক পর্যবেক্ষণ (যেমন temperature বনাম growth rate)।
- **Business Analysis:** বিক্রয় ও বিজ্ঞাপনের খরচের সম্পর্ক বোঝাতে।
- **Data Science & Machine Learning:** variable relationship বিশ্লেষণ এবং regression model validation এ।
- **Health Sector:** weight এবং blood pressure এর সম্পর্ক বিশ্লেষণ করতে।
- **Education:** ছাত্রদের পড়াশোনার সময় বনাম প্রাপ্ত নম্বরের সম্পর্ক বোঝাতে।

Scatter plot সম্পর্ক বোঝা, pattern সনাক্তকরণ, এবং future prediction এর জন্য একটি প্রাথমিক কিন্তু শক্তিশালী visualization tool।

Scatter Plot Basics

Introduction

এই lesson-এ **Scatter Plot** এর মৌলিক ধারণা, কাজ করার পদ্ধতি এবং ব্যবহারিক ক্ষেত্র নিয়ে বিস্তারিত আলোচনা করা হয়েছে। **Scatter Plot** data visualization-এর একটি গুরুত্বপূর্ণ অংশ যা দুটি numerical variable-এর মধ্যে সম্পর্ক প্রদর্শনের জন্য ব্যবহৃত হয়। এটি correlation, pattern, cluster এবং outlier নির্ণয়ে সহায়তা করে।

Theory

Scatter plot হল এমন একটি 2D গ্রাফ যেখানে প্রতিটি point দুটি সংখ্যার (X, Y) মান দ্বারা চিহ্নিত হয়। প্রতিটি observation একটি coordinate হিসেবে দেখা যায়, যা দুটি ভেরিয়েবলের মধ্যে সম্পর্ককে ভিজুয়াল আকারে প্রকাশ করে। Correlation বা সম্পর্কের ধরন সাধারণত তিন প্রকার:

- **Positive Correlation:** যখন X বাড়ে, Y-ও বাড়ে। উদাহরণ: temperature এবং ice cream sales।
- **Negative Correlation:** যখন X বাড়ে, Y কমে। উদাহরণ: speed এবং time (constant distance ক্ষেত্রে)।
- **No Correlation:** X ও Y-এর মধ্যে কোনো স্থির সম্পর্ক নেই। উদাহরণ: shoe size এবং intelligence।

Scatter plot visual form-এ correlation coefficient (r) সম্পর্কে ধারণা দেয় — মান যত +1-এর কাছাকাছি, positive সম্পর্ক তত powerful; যত -1-এর কাছাকাছি, negative সম্পর্ক তত powerful।

Step-by-Step Explanation

Step 1: Import Required Library

```
import matplotlib.pyplot as plt
```

Step 2: Prepare Data

Scatter plot তৈরি করার জন্য দুটি numerical dataset দরকার — X independent variable এবং Y dependent variable।

```
x = [10, 20, 30, 40, 50, 60]
y = [15, 25, 35, 40, 52, 60]
```

Step 3: Create Basic Scatter Plot

`plt.scatter()` function ব্যবহার করে একটি সাধারণ scatter plot তৈরি করা যায়।

```
plt.scatter(x, y)
plt.title('Basic Scatter Plot')
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.grid(True)
plt.show()
```

Step 4: Add Customization

Scatter plot আরও readable এবং presentation-ready করতে বিভিন্ন customization যোগ করা যায়।

```
plt.scatter(x, y, color='blue', s=100, alpha=0.7, edgecolors='black',
marker='o')


plt.title('Customized Scatter Plot Example')
plt.xlabel('Independent Variable (X)')
plt.ylabel('Dependent Variable (Y)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

Customization Parameters:

- **color:** points-এর রঙ পরিবর্তন করে।
- **s:** marker size নির্ধারণ করে।
- **alpha:** transparency নিয়ন্ত্রণ করে।
- **marker:** প্রতিটি point-এর আকার নির্ধারণ করে।
- **edgecolors:** marker-এর border color নির্ধারণ করে।

Step 5: Plot Multiple Datasets

একই plot-এ একাধিক data group প্রদর্শনের জন্য বিভিন্ন color বা marker ব্যবহার করা হয়।

```
x1 = [10, 20, 30, 40, 50]
y1 = [15, 25, 32, 40, 48]

x2 = [10, 20, 30, 40, 50]
y2 = [20, 22, 28, 38, 45]

plt.scatter(x1, y1, color='red', marker='o', label='Group A')
plt.scatter(x2, y2, color='green', marker='s', label='Group B')

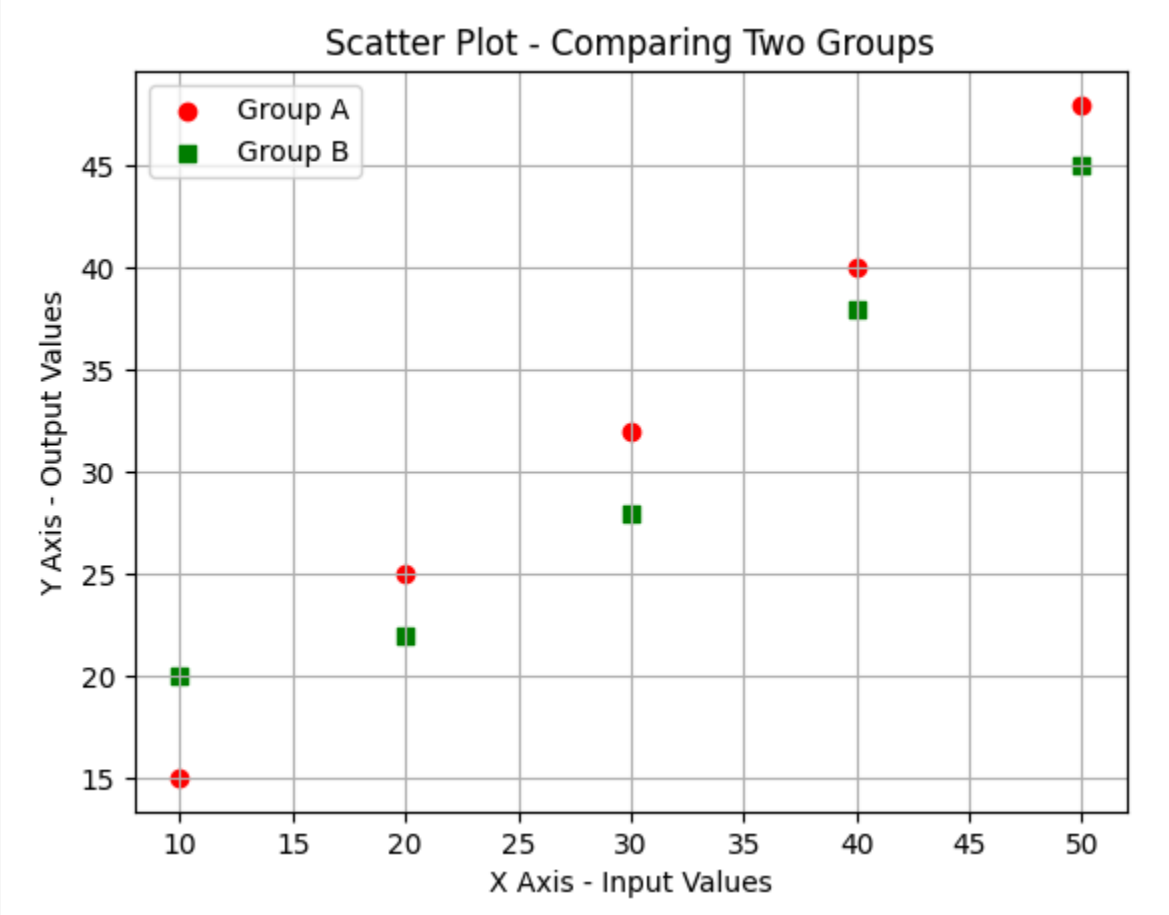
plt.title('Scatter Plot - Comparing Two Groups')
plt.xlabel('X Axis - Input Values')
plt.ylabel('Y Axis - Output Values')
plt.legend()
plt.grid(True)
plt.show()
```

Example Interpretation

এই plot-এ দুটি dataset বিভিন্ন color দ্বারা আলাদা করা হয়েছে। Group A এবং Group B-এর মধ্যে Y-value-এর পরিবর্তন তুলনা করা যায়। যদি দুটি dataset-এর trend প্রায় একই হয়, তবে তাদের মধ্যে positive correlation বোঝা যায়। যদি একটি বাড়ে আর অন্যটি কমে, তবে negative correlation বোঝায়।

Scatter plot outlier সনাক্ত করতেও সহায়ক — যেখানে data points অন্যান্যদের থেকে অনেক দূরে অবস্থান করে।

Output



Output Explanation

Plot-এ প্রতিটি point একটি observation নির্দেশ করে। যত বেশি points একটি diagonal-এর কাছাকাছি থাকে, তত বেশি powerful correlation বোঝায়। Grid যোগ করার ফলে trend আরও স্পষ্টভাবে দেখা যায়।

Practice Question

Task:

একটি dataset দেওয়া আছে যেখানে Temperature (°C) এবং Electricity Usage (kWh) দেওয়া আছে।

```
temp = [10, 15, 20, 25, 30, 35, 40]
usage = [200, 220, 260, 310, 360, 400, 460]
```

একটি scatter plot তৈরি করুন যেখানে temperature X-axis এবং electricity usage Y-axis হিসেবে থাকবে।

title, labels, grid এবং marker style যোগ করুন।

color এবং size পরিবর্তন করে data visualization আরও আকর্ষণীয় করুন।

Challenge:

Scatter plot দেখে Temperature এবং Electricity Usage-এর মধ্যে সম্পর্ক (positive, negative বা none) ব্যাখ্যা করুন।

Histogram

Introduction

এই lesson-এ আলোচনা করা হয়েছে **Histogram**, যা numerical data distribution প্রদর্শনের জন্য সবচেয়ে ব্যবহৃত plotting method। Histogram একটি continuous variable-এর frequency distribution বোঝাতে ব্যবহৃত হয়। এটি মূলত data কীভাবে বিভিন্ন range (বা bins) এ বিভক্ত হয়েছে তা ভিজুয়ালভাবে দেখায়।

Theory

Histogram এবং **Bar Chart** দেখতে প্রায় একই রকম হলেও তাদের মধ্যে কিছু পার্থক্য রয়েছে।

> Histogram continuous data-এর জন্য ব্যবহৃত হয়।
> Bar chart categorical data-এর জন্য ব্যবহৃত হয়।

Histogram-এর X-axis এ থাকে data values বা intervals এবং Y-axis এ থাকে frequency (প্রতিটি bin-এর count)। এটি data distribution, spread, skewness এবং outlier বোঝার জন্য একটি powerful টুল।
Histogram ব্যবহারের মাধ্যমে বোঝা যায়:

> Data symmetric নাকি skewed
> কোথায় বেশি observations রয়েছে (peak)
> Extreme values বা outlier কোথায় আছে

Step-by-Step Explanation

Step 1: Import Required Library

```
1import matplotlib.pyplot as plt
```

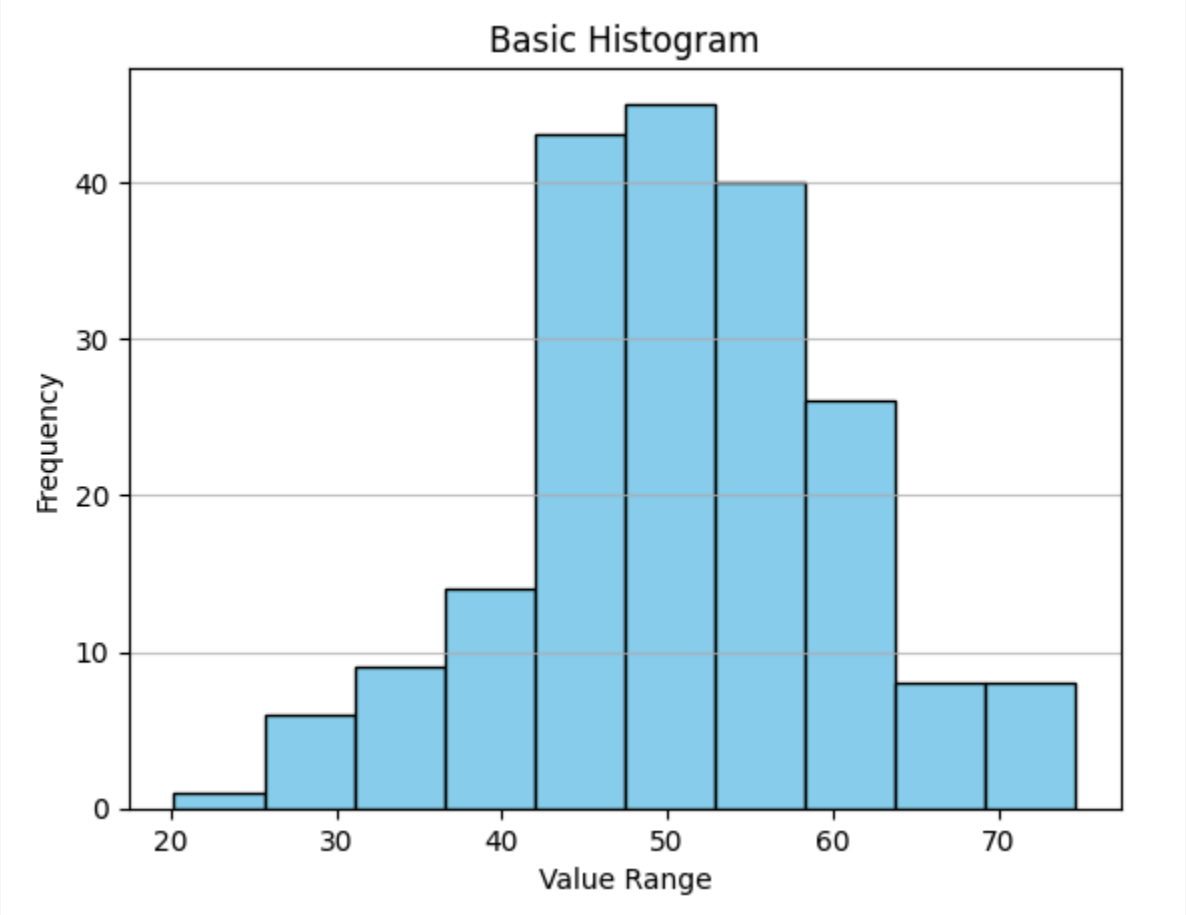
Step 2: Prepare Data

```
1import numpy as np
2
3# Generate random data
4np.random.seed(10)
5data = np.random.normal(50, 10, 200)
```

Step 3: Create a Basic Histogram

```
plt.hist(data, bins=10, color='skyblue', edgecolor='black')
plt.title('Basic Histogram')
plt.xlabel('Value Range')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.75)
plt.show()
```

Output



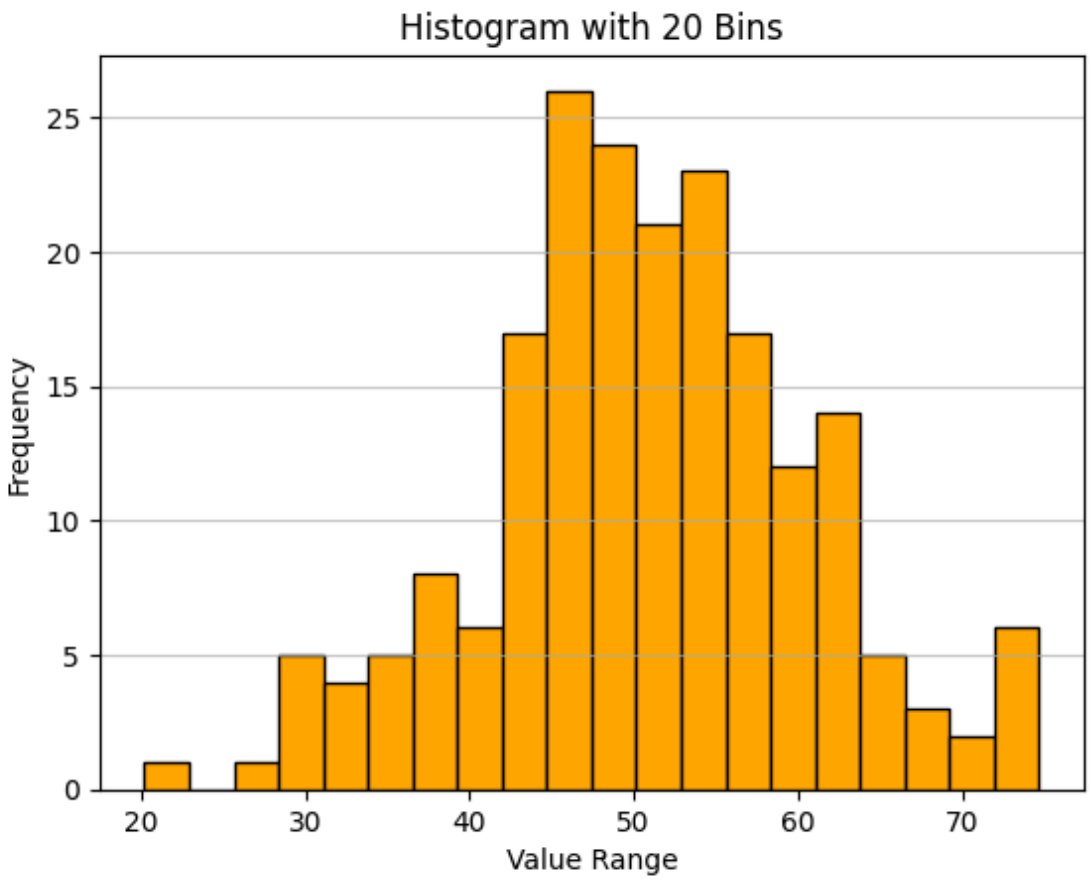
Step 4: Adjust Number of Bins

Bins parameter পরিবর্তন করে histogram-এর granularity পরিবর্তন করা যায়।

```
plt.hist(data, bins=20, color='orange', edgecolor='black')
plt.title('Histogram with 20 Bins')
plt.xlabel('Value Range')
```

```
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.7)
plt.show()
```

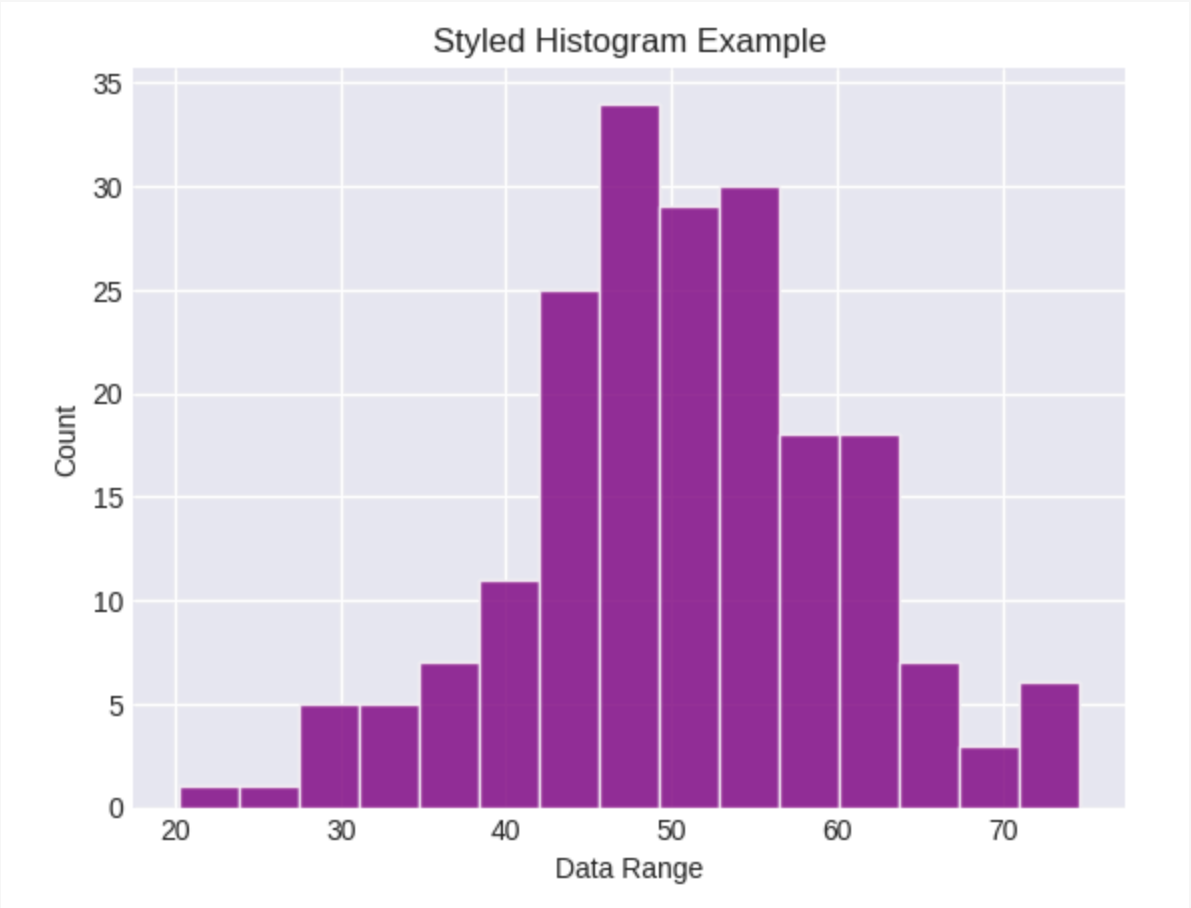
Output



Step 5: Add Style and Customization

```
plt.style.use('seaborn-v0_8-darkgrid')
plt.hist(data, bins=15, color='purple', edgecolor='white', alpha=0.8)
plt.title('Styled Histogram Example')
plt.xlabel('Data Range')
plt.ylabel('Count')
plt.show()
```

Output



Output Explanation

এই Histogram-এ দেখা যায় data-এর majority একটি নির্দিষ্ট range-এর মধ্যে অবস্থান করছে, যা data distribution-এর কেন্দ্র নির্দেশ করে। যদি bar গুলো একদিকে বেশি থাকে তবে dataset skewed বোঝায়। Bins যত ছোট করা হয়, data distribution তত বিস্তারিতভাবে দেখা যায়।

Use Fields

Histogram ব্যবহৃত হয় বিভিন্ন গবেষণা ও বিশ্লেষণমূলক ক্ষেত্রে:

Data Analysis: frequency distribution এবং spread বোঝার জন্য।
Machine Learning: feature distribution এবং normalization যাচাই করতে।
Statistics: data symmetric না skewed তা নির্ধারণে।
Quality Control: measurement error ও defect analysis-এর জন্য।
Finance: price variation বা return distribution বোঝাতে।

Practice Question

Task:

একটি dataset তৈরি করুন যেখানে 300টি random exam score (0 থেকে 100) থাকবে।

- একটি Histogram তৈরি করুন যা score distribution প্রদর্শন করবে।
- bins সংখ্যা পরিবর্তন করে ফলাফল তুলনা করুন।
- title, labels, এবং grid যোগ করুন।

Challenge:
Histogram দেখে score distribution symmetric না skewed তা নির্ণয় করুন।

Bar Chart

Introduction

এই lesson-এ আলোচনা করা হয়েছে **Bar Chart** — একটি অন্যতম জনপ্রিয় visualization পদ্ধতি যা categorical data তুলনা করার জন্য ব্যবহৃত হয়। Bar chart-এর প্রতিটি bar একটি category-র মান (frequency বা measurement) প্রদর্শন করে, যা data-এর তুলনামূলক বিশ্লেষণ সহজ করে।

Theory

Bar Chart মূলত discrete বা categorical data প্রদর্শনের জন্য ব্যবহৃত হয়। প্রতিটি bar-এর দৈর্ঘ্য বা উচ্চতা সেই category-র মান নির্দেশ করে।
Bar Chart সাধারণত দুই প্রকারের হয়:

- **Vertical Bar Chart:** X-axis-এ categories এবং Y-axis-এ মান।
- **Horizontal Bar Chart:** Y-axis-এ categories এবং X-axis-এ মান।

Bar Chart-এর মাধ্যমে:

- একাধিক category-এর মধ্যে তুলনা করা যায়।
- growth, difference বা pattern সহজে বোঝা যায়।
- presentation বা report-এ data স্পষ্টভাবে উপস্থাপন করা যায়।

Step-by-Step Explanation

Step 1: Import Required Library

এই ধাপে **Matplotlib** লাইব্রেরি import করা হয়েছে যা bar chart তৈরি করার জন্য প্রয়োজনীয় plotting functions সরবরাহ করে।

```
1import matplotlib.pyplot as plt
```

Step 2: Prepare Data

এখানে দুটি list তৈরি করা হয়েছে — একটি categories নির্দেশ করছে এবং অন্যটি তাদের মান (values)।

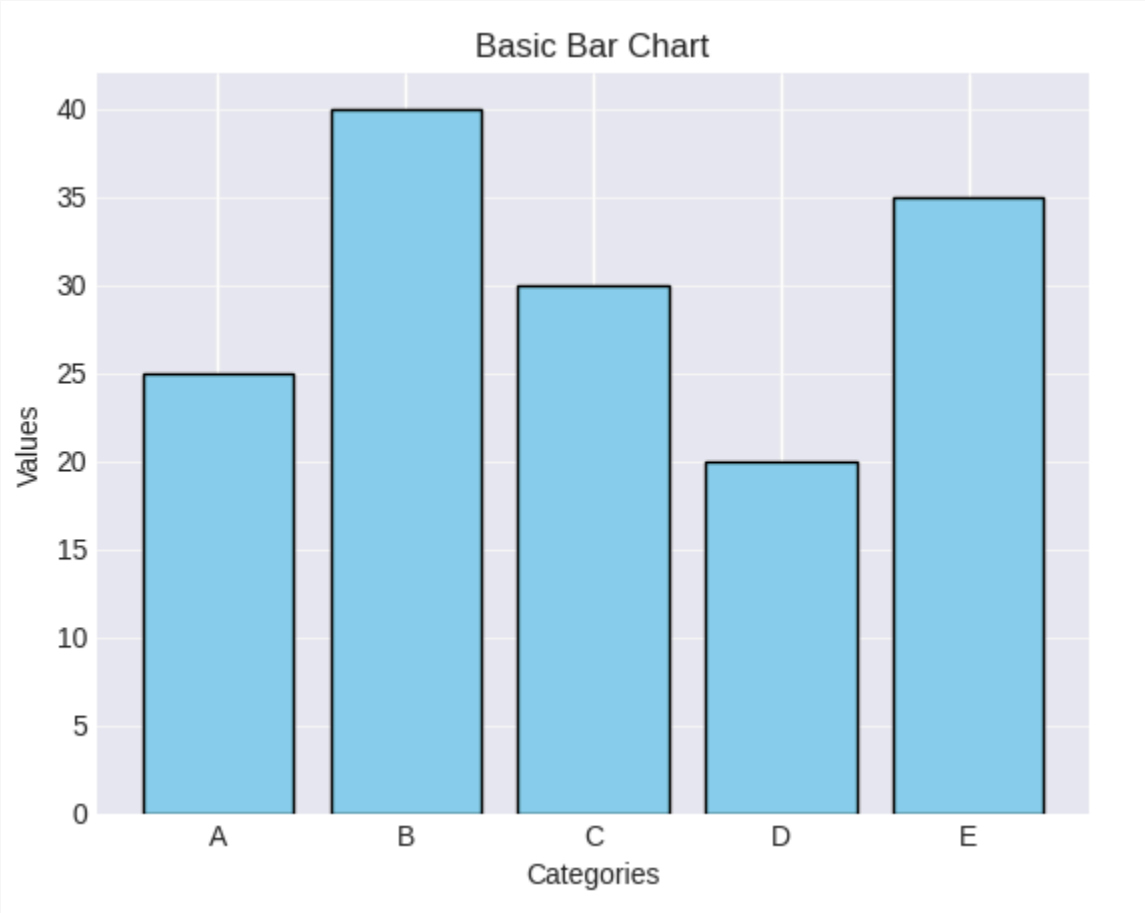
```
categories = ['A', 'B', 'C', 'D', 'E']
values = [25, 40, 30, 20, 35]
```

Step 3: Create Basic Vertical Bar Chart

এই ধাপে একটি সাধারণ vertical bar chart তৈরি করা হয়েছে যা প্রতিটি category-এর মান প্রদর্শন করে।

```
plt.bar(categories, values, color='skyblue', edgecolor='black')
plt.title('Basic Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.grid(axis='y', alpha=0.6)
plt.show()
```

Output

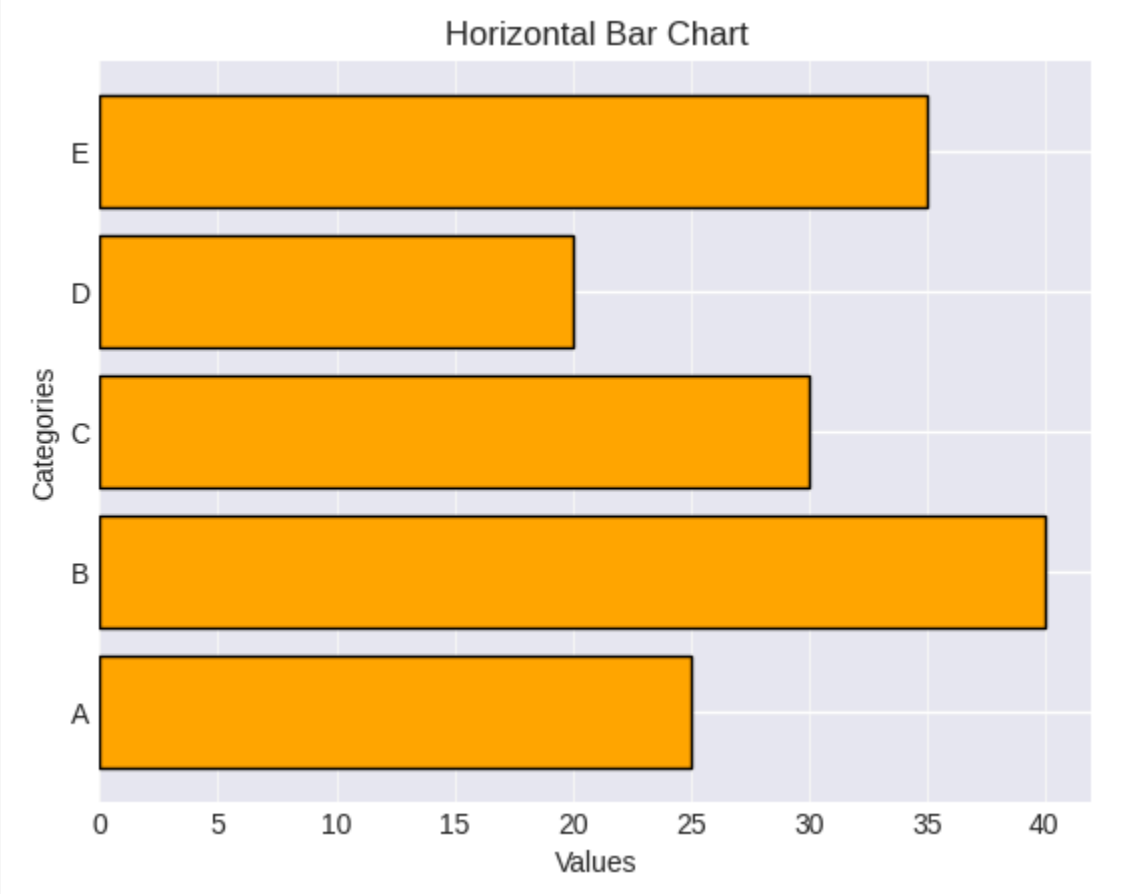


Step 4: Create Horizontal Bar Chart

এখানে একই data horizontal আকারে প্রদর্শন করা হয়েছে, যেখানে category গুলো Y-axis এ থাকে। এটি দীর্ঘ নামের category প্রদর্শনের জন্য উপযোগী।

```
plt.barh(categories, values, color='orange', edgecolor='black')
plt.title('Horizontal Bar Chart')
plt.xlabel('Values')
plt.ylabel('Categories')
plt.grid(axis='x', alpha=0.6)
plt.show()
```

Output



Step 5: Add Multiple Bars (Comparison)

এই ধাপে দুটি dataset একসাথে প্রদর্শন করা হয়েছে যাতে 2024 এবং 2025 সালের data তুলনা করা যায়।

```
import numpy as np

x = np.arange(len(categories))
```

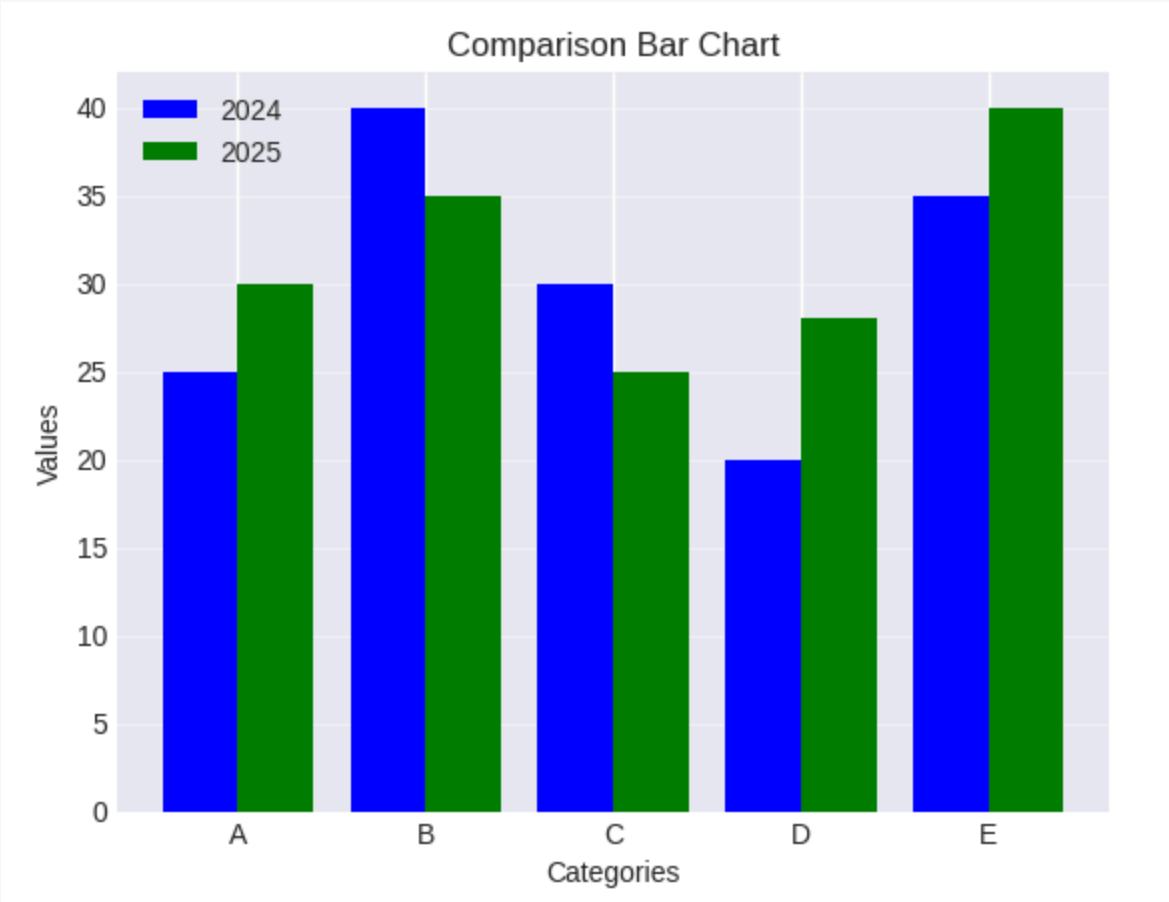
```
value1 = [25, 40, 30, 20, 35]
value2 = [30, 35, 25, 28, 40]

plt.bar(x - 0.2, value1, width=0.4, color='blue', label='2024')
plt.bar(x + 0.2, value2, width=0.4, color='green', label='2025')

plt.xticks(x, categories)
plt.title('Comparison Bar Chart')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.legend()
plt.grid(axis='y', alpha=0.5)
plt.show()
```

এই plot থেকে সহজেই দেখা যায় দুটি বছরের data-এর মধ্যে কোন category তে পরিবর্তন ঘটেছে।

Output



Output Explanation

Vertical bar chart-এ প্রতিটি bar একটি category-র মান বোঝায়। Multiple bar chart ব্যবহার করলে একই category-এর জন্য দুই বা ততোধিক dataset তুলনা করা যায়। Horizontal chart ব্যবহার করলে দীর্ঘ নামের categories সহজে প্রদর্শন করা যায়।

Fields of Use

Business Analytics: product-wise sales comparison।
Education: subject-wise student performance analysis।
Finance: yearly profit/loss visualization।
Survey Analysis: জনমত বা feedback ফলাফল প্রদর্শন।
Research: বিভিন্ন experimental group-এর তুলনামূলক ফলাফল দেখাতে।

Practice Question

Task:
একটি dataset রয়েছে যেখানে তিনটি brand-এর monthly sales দেওয়া আছে:

```
brands = ['Brand A', 'Brand B', 'Brand C']
sales_2024 = [250, 400, 350]
sales_2025 = [300, 420, 370]
```

- দুটি বার চার্ট তৈরি করুন — একটি vertical এবং একটি horizontal।
- দুটি বছরের data একই chart-এ প্রদর্শন করুন।
- title, labels, legend, এবং grid যোগ করুন।

Challenge:
Bar chart দেখে নির্ণয় করুন কোন brand-এর বিক্রি ২০২৫ সালে সর্বাধিক বৃদ্ধি পেয়েছে।

Pie Chart

Introduction

এই lesson-এ আলোচনা করা হয়েছে **Pie Chart**, যা একটি জনপ্রিয় visualization technique যেখানে data proportion বা percentage আকারে প্রদর্শন করা হয়। Pie chart ব্যবহার করে একটি dataset-এর বিভিন্ন অংশের মধ্যে সম্পর্ক বা ভাগ সহজে বোঝা যায়। প্রতিটি slice একটি category-র অংশ (portion) নির্দেশ করে।

Theory

Pie Chart data visualization-এর এমন একটি পদ্ধতি যেখানে মোট মানকে 100% ধরে প্রতিটি অংশকে একটি বৃত্তের নির্দিষ্ট কোণে ভাগ করা হয়।
Pie Chart-এর প্রধান বৈশিষ্ট্য:

- প্রতিটি slice-এর আকার তার মানের (value) অনুপাতে নির্ধারিত হয়।
- এটি data-এর composition বোঝাতে ব্যবহার করা হয়।
- Pie Chart একবারে সীমিত সংখ্যক category (৪–৬) প্রদর্শনের জন্য উপযুক্ত।

Pie chart সাধারণত percentage বা share বোঝানোর জন্য ব্যবহৃত হয় যেমন — market share, survey response distribution, বা budget allocation।

Step-by-Step Explanation

Step 1: Import Required Library

```
import matplotlib.pyplot as plt
```

Step 2: Prepare Data

```
labels = ['A', 'B', 'C', 'D']
```

```
values = [30, 25, 20, 25]
```

Step 3: Create a Basic Pie Chart

এই ধাপে একটি সাধারণ pie chart তৈরি করা হয়েছে যেখানে প্রতিটি category-এর অংশ দেখা যায়।

```
plt.pie(values, labels=labels)
plt.title('Basic Pie Chart')
plt.show()
```

Step 4: Add Customization

Pie Chart-কে আরও informative করতে percentage, colors, এবং highlighting যোগ করা যায়।

```
colors = ['gold', 'lightcoral', 'lightskyblue', 'yellowgreen']
explode = [0.05, 0, 0, 0]  # প্রথম slice কে আলাদা করে দেখানো হয়েছে

plt.pie(values, labels=labels, colors=colors, autopct='%1.1f%%',
startangle=90, explode=explode, shadow=True)
plt.title('Customized Pie Chart')
plt.show()
```

Step 5: Add Legend

Pie chart-এ legend ব্যবহার করে প্রতিটি slice-এর নাম বা মান আরও স্পষ্টভাবে দেখানো যায়।

```
plt.pie(values, labels=labels, colors=colors, autopct='%1.1f%%')
plt.legend(title='Categories')
plt.title('Pie Chart with Legend')
plt.show()
```

Output Explanation

এই chart-এ দেখা যায় প্রতিটি category-এর অংশ বৃত্তের নির্দিষ্ট অংশ দখল করেছে। **autopct** parameter percentage দেখায়, আর **explode** parameter একটি slice আলাদা করে focus করতে সাহায্য করে। Pie chart থেকে সহজেই বোঝা যায় কোন অংশ সবচেয়ে বড় বা ছোট।

Fields of Use

Business: market share বা revenue distribution বোঝাতে।

Finance: budget allocation বা expense breakdown প্রদর্শন করতে।

Survey Analysis: response percentage visualization।

Education: student group participation বা performance ratio দেখাতে।

Research: data proportion বা population distribution প্রদর্শনের জন্য।

Practice Question

Task:

একটি dataset রয়েছে যেখানে চারটি product-এর market share দেওয়া আছে:

```
products = ['Product A', 'Product B', 'Product C', 'Product D']
share = [40, 25, 20, 15]
```

একটি pie chart তৈরি করুন যা market share প্রদর্শন করবে।

percentage label, color, এবং explode parameter ব্যবহার করুন।

title এবং legend যোগ করুন।

Challenge:

Pie chart দেখে নির্ণয় করুন কোন product-এর market share সবচেয়ে বেশি এবং কোনটি সবচেয়ে কম।# 15.8 Bar